

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 – Программная инженерия, Системное
и прикладное программное обеспечение

Дисциплина: Информационные системы

Лабораторная работа №1
Вариант №55995

Выполнил:
Карнажицкий Максим Романович
Группа: Р3311

Проверил:
Тюрин И. Н.

г. Санкт-Петербург, 2026

Содержание

1. Задание	2
2. Диаграмма классов разработанного приложения	6
3. Исходный код	7
3.1. Ссылка на репозиторий	7
3.2. Комментарии к реализации	7
3.2.1. Auth	7
3.2.2. Server Sent Events	7
3.2.3. Общее по фронтенду	8
4. Вывод	10

1. Задание

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса `Movie`, описание которого приведено ниже:

```
public class Movie {
    private Integer id; //Поле не может быть null, Значение поля должно быть больше 0, Значение
этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null, Значение этого поля должно
генерироваться автоматически
    private Long oscarsCount; //Значение поля должно быть больше 0
    private Double budget; //Значение поля должно быть больше 0, Поле не может быть null
    private Long totalBoxOffice; //Значение поля должно быть больше 0
    private Mpaarating mpaaRating; //Поле может быть null
    private Person director; //Поле не может быть null
    private Person screenwriter;
    private Person operator; //Поле может быть null
    private Long length; //Поле может быть null, Значение поля должно быть больше 0
    private int goldenPalmCount; //Значение поля должно быть больше 0
    private MovieGenre genre; //Поле может быть null
}
public class Coordinates {
    private Long x; //Максимальное значение поля: 455, Поле не может быть null
    private int y; //Значение поля должно быть больше -124
}
public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Color eyeColor; //Поле может быть null
    private Color hairColor; //Поле не может быть null
    private Location location; //Поле может быть null
    private Double height; //Поле не может быть null, Значение поля должно быть больше 0
    private Country nationality; //Поле может быть null
}
public class Location {
    private double x;
    private long y;
    private Float z; //Поле не может быть null
    private String name; //Длина строки не должна быть больше 389, Поле может быть null
}
public enum Mpaarating {
    PG_13,
    R,
    NC_17;
}
public enum MovieGenre {
    WESTERN,
    TRAGEDY,
    THRILLER,
    HORROR,
    FANTASY;
}
public enum Color {
    GREEN,
    BLACK,
    YELLOW,
    ORANGE;
}
public enum Country {
    FRANCE,
    INDIA,
    VATICAN,
```

```
SOUTH_KOREA,  
NORTH_KOREA;  
}
```

Разработанная система должна удовлетворять следующим требованиям:

- Основное назначение информационной системы - управление объектами, созданными на основе заданного в варианте класса.
- Необходимо, чтобы с помощью системы можно было выполнить следующие операции с объектами: создание нового объекта, получение информации об объекте по ИД, обновление объекта (модификация его атрибутов), удаление объекта. Операции должны осуществляться в отдельных окнах (интерфейсах) приложения. При получении информации об объекте класса должна также выводиться информация о связанных с ним объектах.
- При создании объекта класса необходимо дать пользователю возможность связать новый объект с объектами вспомогательных классов, которые могут быть связаны с созданным объектом и уже есть в системе.
- Выполнение операций по управлению объектами должно осуществляться на серверной части (не на клиенте), изменения должны синхронизироваться с базой данных.
- На главном экране системы должен выводиться список текущих объектов в виде таблицы (каждый атрибут объекта - отдельная колонка в таблице). При отображении таблицы должна использоваться пагинация (если все объекты не помещаются на одном экране).
- Нужно обеспечить возможность фильтровать/сортировать строки таблицы, которые показывают объекты (по значениям любой из строковых колонок). Фильтрация элементов должна производиться по неполному совпадению.
- Переход к обновлению (модификации) объекта должен быть возможен из таблицы с общим списком объектов и из области с визуализацией объекта (при ее реализации).
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться в интерфейсах у других пользователей, авторизованных в системе.
- Если при удалении объекта с ним связан другой объект, операция должна быть отменена, пользователю нужно сообщить о невозможности удаления объекта.
- Пользователи должны иметь возможность просмотра всех объектов. Для модификации объекта должно открываться отдельное диалоговое окно.

При вводе некорректных значений в поля объекта должны появляться информативные сообщения о соответствующих ошибках.

В системе должен быть реализован отдельный пользовательский интерфейс для выполнения специальных операций над объектами:

- Сгруппировать объекты по значению поля `oscarsCount`, вернуть количество элементов в каждой группе.
- Вернуть массив объектов, значение поля `name` которых содержит заданную подстроку.
- Вернуть массив объектов, значение поля `goldenPalmCount` которых меньше заданного.
- Получить список фильмов, не имеющих ни одного «Оскара».
- Дополнительно наградить все фильмы с продолжительностью больше заданной указанным числом «Оскаров».
- Представленные операции должны быть реализованы в рамках компонентов бизнес-логики приложения без прямого использования функций и процедур БД.

Особенности хранения объектов, которые должны быть реализованы в системе:

- Организовать хранение данных об объектах в реляционной СУБД (PostgreSQL). Каждый объект, с которым работает ИС, должен быть сохранен в базе данных.
- Все требования к полям класса (указанные в виде комментариев к описанию классов) должны быть выполнены на уровне ORM и БД.
- Для генерации поля `id` использовать средства базы данных.
- Для подключения к БД на кафедральном сервере использовать хост `pg`, имя базы данных - `studs`, имя пользователя/пароль совпадают с таковыми для подключения к серверу.

При создании системы нужно учитывать следующие особенности организации взаимодействия с пользователем:

- Система должна реагировать на некорректный пользовательский ввод, ограничивая ввод недопустимых значений и информируя пользователей о причине ошибки.
- Переходы между различными логически обособленными частями системы должны осуществляться с помощью меню.
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться на области `u` всех других клиентов.

- При разработке ИС должны учитываться следующие требования:

В качестве основы для реализации ИС необходимо использовать Spring MVC. Для создания уровня хранения необходимо использовать JPA + Hibernate. Разные уровни приложения должны быть отделены друг от друга, разные логические части ИС должны находиться в отдельных компонентах.

2. Диаграмма классов разработанного приложения

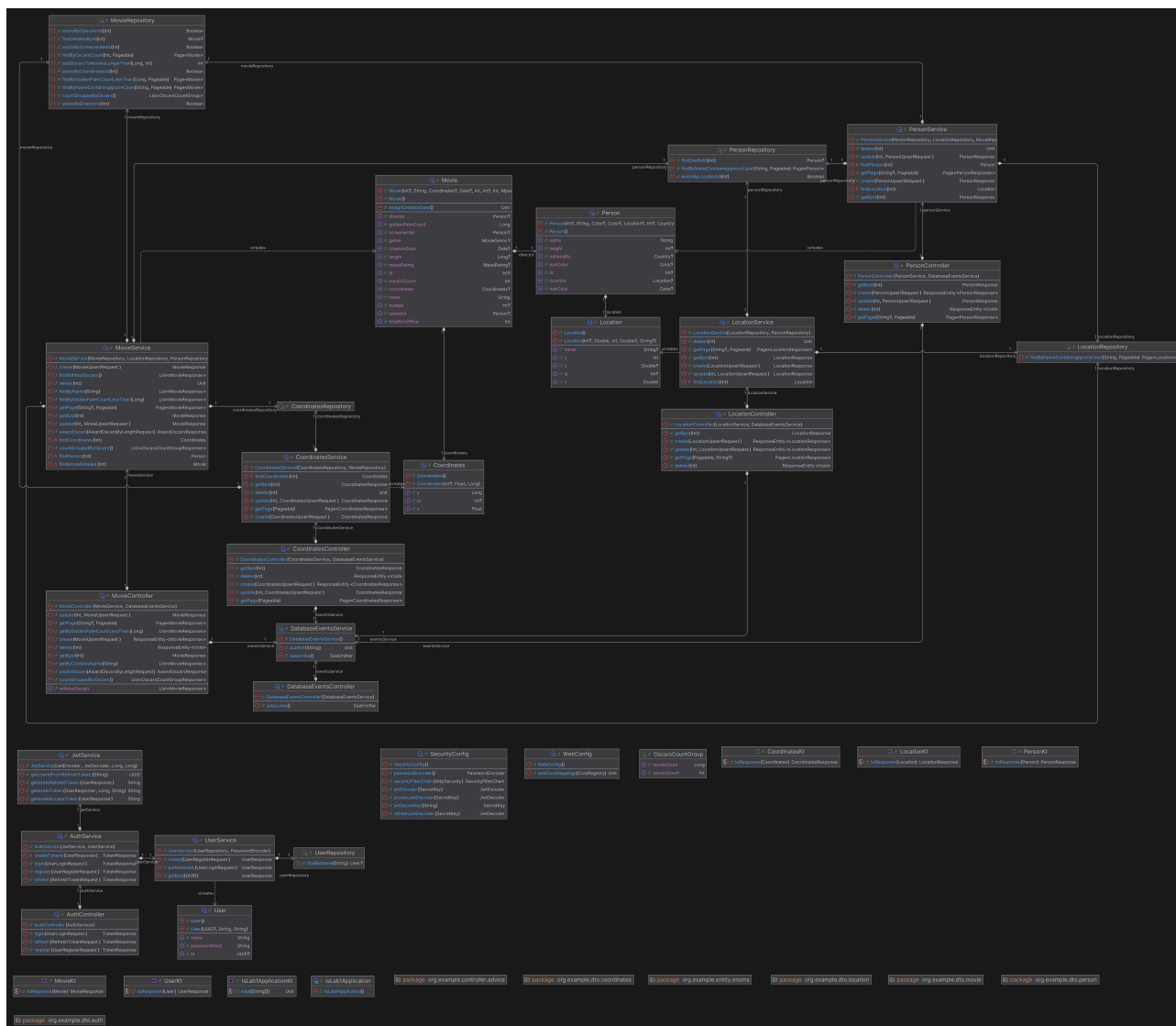


Рис. 1. UML-диаграмма классов (кроме DTO)

3. Исходный код

3.1. Ссылка на репозиторий

https://github.com/xtern0o/is-lab1/blob/main/is_lab1

3.2. Комментарии к реализации

3.2.1. Auth

Я использовал самую простую `stateless jwt access+refresh` для авторизации и аутентификации пользователей.

Сам `User entity` в себе хранит только `name`, `passwordHash` – остальное в рамках лабы, как по мне, избыточно (по сути нам нужно просто выдавать доступ ко всему API)

Генерация JWT – в компоненте `service/JwtService`. Из тонкостей: сделал 2 отдельных бина `JwtDecoder` для `access/refresh` токенов, так как иначе можно было передать `refresh` токен в `Bearer` заголовке, и это прокатывало.

`AuthService` отвечает за высокоуровневую бизнес-логику авторизации: регистрация, логин и обновление токенов, дергая методы `UserService` и `JwtService`. Именно с ним связан `AuthController`.

В `SecurityConfig` я указал использование `oauth2ResourceServer jwt`. По сути это просто упростило мне работу с `jwt flow`: Spring нативно проверяет поступающий `Bearer: <Auth Token>` на предмет корректности, благодаря чему мне не надо писать `servlet filters`.

3.2.2. Server Sent Events

Так как задание требует своевременного обновления данных на всех клиентах, необходимо было выбрать свой стул:

1. Short polling
2. Long polling
3. WebSocket
4. SSE

Я выбрал SSE, потому что ивенты обновления базы данных отсылаются в одностороннем порядке (отмечаем `WebSocket`), и в целом не так сильно нагружает Бд, как `ShortPolling`. А `Long polling` – это просто костыль SSE.

Для реализации не бекенде я использовал встроенные в `spring.mvc` инструменты. Из тонкостей:

- `DatabaseEventsService` содержит в себе `CopyOnWriteArrayList<SseEmitter>` - это потому, что подключения и отключения могут приходить из разных потоков. Так мы синхронизируем данные.

По сути реализация следующая:

- При успешном обновлении какой-либо таблицы, всем клиентам отсылается событие вида: `updated: {entity}`, где `entity` – какая-то строка, название таблицы, которую отредактировали.
- Фронтенд, получая это событие, делает запрос на получение текущей страницы **только в том случае**, если `pinia state` текущего энтита (просматриваемой таблицы) совпадает с пришедшим событием на обновление (иначе бессмысленно). Таким образом обновляется текущая просматриваемая страница.
- На фронтенде я использовал `@microsoft/fetch-event-source` лишь потому, что с обычным браузерным `EventSource` я не понял, как отослать `header Bearer: <Auth>`. А запросы на обновление может слать только авторизованный пользователь (по заданию)!!

3.2.3. Общее по фронтенду

Использовал **Vue.js 3**

Состояние всего приложения хранится в `pinia` для централизованного управления.

Токены хранятся в `localStorage`. **Понимаю риски утечки** - это допущение в рамках лабораторной.

Разработана структура энтита для гибкой поддержки отрисовки.

Где использовался ИИ:

- Сами `ENTITY_DEFINITIONS` (описание энтита как таблиц, которыми мы оперируем) сгенерировано ИИ, так как работа очень простая и незамысловатая. Результат - файл `@frontend/src/components/domain/entitySchemas.ts`

▸ Prompt:

На основе backend сущностей сгенерируй TypeScript-константу ``ENTITY_DEFINITIONS`` для описания сущностей во frontend.

это не сами данные и не классы сущностей, а метаданные для автоматического построения таблиц и форм редактирования

Структура каждой сущности:

- ключ объекта (`movies`, `persons`, `locations`) - идентификатор вкладки
- `typeName` - отображаемое название сущности
- `statusNoun` -подпись количества записей
- `searchableKeys` -поля, по которым разрешён текстовый поиск
- `columns` -поля сущности в порядке отображения

Описание колонки:

- `key` -название поля из JSON backend;
- `label` -подпись в интерфейсе;
- `readonly: true` -поле нельзя редактировать, например `id` или `creationDate`;
- `relation` -название связанной сущности, если поле содержит внешний ID;
- `options` -массив допустимых значений для enum-поля.

Связи должны представляться как ID: например, вместо объекта `director` используется `directorId` с `relation: 'persons'`.

```

вот смотри пример {
  key: 'directorId',
  label: 'DIRECTOR ID',
  relation: 'persons'
}

```

```

enum-поле {
  key: 'genre',
  label: 'GENRE',
  options: GENRES
}

```

Не добавляй вымышленные поля. Используй точные названия полей из dto, прикрепленные ниже.

[вставленные сущности dto]

- большая часть css-стилей также была сгенерирована ИИ с целью экономии времени

4. Вывод

В ходе лабораторной работы я создал круд-приложение с несколькими сущностями, а также специальными операциями над ними. Вспомнил, до чего же утомительно писать фронтенд. Вспомнил `jwt flow`, а также потыкал SSE.